

Kotlin coroutines

A complete study guide, from syntax to resilient Android code

Build a precise mental model of suspension, task ownership, cancellation, and failure. Then apply it to streams, Android lifecycles, and deterministic tests.

Your anchor problem

```
suspend fun fetchProfile(): String {
    delay(100)
    return "profile"
}
suspend fun fetchOrders(): String {
    delay(50)
    throw IllegalStateException("Wrong orders")
}
```

Why can orders cancel profile before orders.await() is reached? How can profile finish anyway? Every chapter adds a piece of the answer, then asks you to predict and test the behavior.

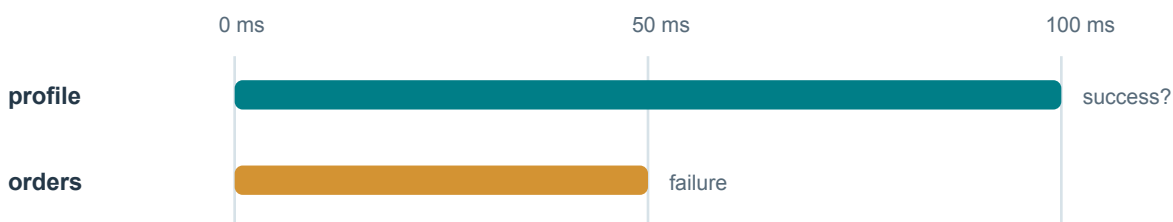
35 study sessions / 5 weeks

Plan for 45-60 minutes per study day. Read, predict, run, change one variable, and explain the result. The schedule includes implementation days, review days, and a capstone.

Prepared for the continuation of “Analyze Kotlin Syntax.” Examples and exercises use the original profile/orders scenario. Reviewed edition / 30 September 2026.

Code convention: JVM snippets use `kotlinx.coroutines.*`; Flow snippets also use `kotlinx.coroutines.flow.*`. Android and test listings identify their extra imports. Keep functions/types at file scope; put loose statements inside main or a test. Page 35 explains placement and test setup.

THE QUESTION THAT DRIVES THE GUIDE



How to use this guide

Learn the rules, then prove them with small experiments

Read	Topics	Pages
Foundations	Setup, Kotlin syntax, threads, suspension	3-7
Task ownership	Builders, original program, Job tree, scopes	8-11
Failure and cancellation	Cancellation, cleanup, exceptions, supervision	12-16
Choosing the tool	Dispatchers, async vs withContext, error models	17-20
Streams	Flow operators, StateFlow, SharedFlow	21-24
Android and tests	ViewModel, lifecycle collection, virtual time	25-28
Practice	5-week schedule, labs, answers, quick reference	29-34
Worked practice	Test setup, solutions, races, Android tests	35-40
Official resources	Annotated, clickable reading list	41-42

A session that builds understanding

Spend 10 minutes reading, 10 minutes predicting the next output and drawing the Job tree, 20-30 minutes running and modifying the example, and 5-10 minutes writing a short explanation from memory. Record expected output before pressing Run.

Use approximate timelines only for reasoning. The 50 ms and 100 ms delays are teaching fixtures, not promises about wall-clock scheduling. Tests later replace those delays with virtual time.

Definition of progress

You can name the owner of each coroutine, explain how cancellation travels, distinguish failure from cancellation, and choose a scope based on the desired product behavior.

If a checkpoint fails, repeat the related experiment with fewer moving parts. You can accelerate the syntax chapters if they are familiar, but keep the exception and supervision exercises.

Set up a small Kotlin lab

A runnable JVM starting point before Android integration

Create a Kotlin/JVM Gradle project in IntelliJ IDEA. Keep its generated Kotlin plugin and JDK configuration. Add the following repositories and dependencies to `build.gradle.kts`, then sync Gradle. The official coroutine documentation currently shows 1.11.0; keep core and test on the same version. [1, 22]

```
repositories { mavenCentral() }
dependencies {
    implementation(
        "org.jetbrains.kotlinx:kotlinx-coroutines-core:1.11.0"
    )
    testImplementation(kotlin("test-junit5"))
    testImplementation(
        "org.jetbrains.kotlinx:kotlinx-coroutines-test:1.11.0"
    )
}
```

Put this in `src/main/kotlin/Main.kt`. It is a complete, successful first run. Keep `fetchOrders` failing; create a separate successful fixture when an exercise asks for one.

```
import kotlinx.coroutines.*

suspend fun fetchProfile(): String {
    delay(100)
    return "profile"
}

suspend fun fetchOrders(): String {
    delay(50)
    throw IllegalStateException("Wrong orders")
}

fun main() = runBlocking {
    println(fetchProfile())
}
```

Expected output: profile. If delay is unresolved, first check the coroutine dependency, Gradle sync, and the import. A Kotlin file alone does not supply `kotlinx.coroutines`. If the chosen coroutine version requires a newer Kotlin compiler, update the project using the official compatibility information rather than mixing arbitrary versions.

Keep examples separate

Replace main for each experiment. Do not paste several main functions with identical signatures into one source set. Android dependencies appear in its chapters. Before running tests, add the test engine configuration on page 35.

Read Kotlin syntax confidently

Functions, values, types, nullability, and exceptions

A function declares a name, inputs, and an output type. A block body uses `return` to produce its result; an expression body uses the expression after `=`. `val` prevents reassignment of the reference, while `var` allows it. Neither makes a referenced mutable object thread-safe.

```
fun label(id: Int): String { return "user-$id" }
fun labelShort(id: Int): String = "user-$id"
val profile: String = label(7)
var attempts = 0
attempts += 1
println("profile: $profile, attempts: $attempts")
```

String interpolation evaluates `$name` or the expression inside `${...}`. In your original `println`, the `profile` await expression is evaluated before the `orders` await expression. That expression order does not control when the two async computations start or fail.

Failure as control flow

```
fun ordersLabel(ok: Boolean): String {
    if (!ok) throw IllegalStateException("Wrong orders")
    return "orders"
}
val message = try {
    ordersLabel(false)
} catch (e: IllegalStateException) {
    "unavailable: ${e.message}"
}
```

`try` can be an expression. A `catch` handles a matching exception thrown while that call stack executes. A `finally` block runs when leaving the `try`, including during exception propagation. Throwing interrupts the current path; code after the throw is unreachable. [3]

`String?` allows null; `String` does not. A nullable result can mean “absent,” but cannot by itself distinguish missing data from a network failure. `Result<String>` later represents success or failure explicitly. Generic types such as `List<String>` and `Deferred<String>` say what kind of value a container holds.

Exercise 1

Write a function that returns a label, then one that throws. Predict which lines execute inside `try/catch/finally`. Explain why `catch` does not undo work already completed.

Lambdas, receivers, and generics

Why `launch { ... }` is valid Kotlin syntax

A lambda is a function value. `() -> String` describes a function taking no arguments and returning `String`. A function accepting another function is a higher-order function. Kotlin allows a final lambda argument to appear outside parentheses. [2]

```
fun describe(load: () -> String): String = load()
val answer = describe { "profile" }

class Notebook { fun line(s: String) = println(s) }
fun write(block: Notebook.() -> Unit) {
    Notebook().block()
}
write {
    line("Implicit receiver")
    this.line("Explicit receiver")
}
```

`Notebook.() -> Unit` is a lambda with a receiver. Inside it, `this` is a `Notebook`, so `line` resolves without an explicit object. Coroutine builders similarly provide a `CoroutineScope` receiver. `launch` and `async` are extension functions on that receiver.

```
fun CoroutineScope.startProfile(): Job = launch {
    println(fetchProfile())
}
// Approximate block type accepted by async:
// suspend CoroutineScope.() -> T
// async { fetchProfile() } produces Deferred<String>.
```

The extension receiver gives a function access to a scope; it does not inherit from that class or create a new scope. `suspend` in a lambda type permits suspension inside the lambda. `T` is a generic type parameter, and the compiler infers `String` from `fetchProfile`. `Deferred<T>` combines a lifecycle handle with a future result.

Exercise 2

Rewrite `launch` as `this.launch` inside `runBlocking`. Then move `launch` into an ordinary function with no scope receiver: observe the unresolved reference. Fix it by accepting a `CoroutineScope` or declaring an extension receiver. Decide who owns the passed scope.

Official reading: [2] Lambdas; [1] Coroutines basics. Sources are linked on pages 41-42.

Threads, blocking, and concurrency

Separate execution resources from the work scheduled on them

A JVM thread executes instructions. Blocking occupies that thread while waiting: `Thread.sleep` and a synchronous file read are examples. A coroutine can suspend its work so the thread runs other work, provided the operation actually supports suspension. A coroutine still needs a thread when executing code.

Idea	Meaning in the profile/orders lab
Sequential	Start and finish profile, then start orders.
Concurrent	Both tasks are in progress over overlapping intervals.
Parallel	Instructions execute simultaneously on different CPU cores.
Blocking	A waiting operation holds a thread.
Suspending	A waiting coroutine yields its thread for other work.

```
fun main() = runBlocking {
    launch { delay(100); println("A") }
    launch { delay(50); println("B") }
    println("started")
}
```

With this inherited `runBlocking` event loop, the delayed work overlaps on one thread: started, B, A. Parallel CPU execution is unnecessary. Change the first delay to `Thread.sleep(100)`: once that child runs, it prevents the same thread from running the second child until sleep returns. This alters responsiveness and timing, not just spelling.

Concurrency introduces shared-state problems

Two coroutines on a multithreaded dispatcher can race on `counter++`. Suspension can also interleave read-then-write logic on a single thread. Prefer immutable results, confined ownership, atomic operations, or `Mutex` for coordinated mutation. Keeping a variable as `val` does not protect the contents of a mutable list.

Exercise 3

Draw one thread as a lane and mark when each coroutine runs or waits. Explain why two delay calls can overlap, while a single thread cannot execute two CPU instructions at the same instant.

Official reading: [1] Basics; [16] Context and dispatchers. Sources are linked on pages 41-42.

What suspend does

A function may pause; it does not automatically run elsewhere

`suspend` changes where a function can be called and permits suspension. It does not start a coroutine, pick a dispatcher, make the body concurrent, or convert blocking calls to nonblocking calls. A suspend function can finish without ever suspending.

```
suspend fun fetchBothSequential(): Pair<String, String> {  
    val profile = fetchProfile()  
    val orders = fetchOrders()  
    return profile to orders  
}
```

This calls profile first. About 100 ms later it starts orders, which fails after another 50 ms. No independent task was started. Sequential syntax remains sequential until a builder explicitly introduces concurrent work. [17]

A useful implementation model

The compiler transforms suspension into resumable state: local values and the next execution position can be retained across pauses. A continuation represents what happens next. You do not need to implement this machinery to use it, but it explains why a function can resume without keeping a thread blocked throughout its delay.

```
suspend fun misleading() {  
    Thread.sleep(100) // Still blocks the executing thread.  
}  
suspend fun cooperative() {  
    delay(100) // Cancellable suspending wait.  
}
```

Call suspend functions from a suspend function or a coroutine builder body. In console exercises, `runBlocking` bridges main to suspend code. In Android, start work in a lifecycle-owned scope. Later, `withContext` moves blocking work to an appropriate dispatcher.

Checkpoint A

Can you explain why `fetchProfile()` is sequential when called directly, why `delay` is a library function, and why `suspend` alone does not mean “background thread”? If not, repeat pages 3-7 before adding failure policies.

Official reading: [1] Basics; [17] Composing suspending functions. Sources are linked on pages 41-42.

Four essential entry points

Understand the result, waiting behavior, and intended use

API	Returns	How the caller continues
launch	Job	Starts a task; caller can continue. join waits.
async	Deferred<T>	Starts a computation; await waits for its value.
runBlocking	T	Blocks its calling thread until its block and children finish.
withContext	T	Suspends until its block and children finish; uses the supplied context.

```
fun main() = runBlocking {
    val job: Job = launch { println(fetchProfile()) }
    val deferred: Deferred<String> = async { fetchProfile() }
    println("children started")
    job.join()
    println(deferred.await())
    val value = withContext(Dispatchers.Default) { 2 + 3 }
    println(value)
}
```

join observes completion without retrieving a result or rethrowing that Job's failure. It can still throw cancellation if the waiting coroutine is cancelled. await retrieves the result or throws; it also respects cancellation of the waiting coroutine. A normal child failure may already cancel the waiter's parent before either operation.

launch is useful for UI-triggered operations when the value is delivered through state. async is useful when several concurrent values must be combined. withContext is useful for a single result computed in a different context. Use runBlocking at a blocking boundary such as these console demos; never to make Android UI code wait for a request.

Start behavior

The default start schedules the coroutine according to its dispatcher; it does not guarantee its first line runs before the builder returns. LAZY defers starting until start, await, or join. UNDISPATCHED and other modes are advanced tools, not fixes for exception handling.

Official reading: [1] Basics; [11] async; [12] withContext; [10] Job. Sources are linked on pages 41-42.

Decode your original program

The catch surrounds starting a task, not its later execution

```
fun main() = runBlocking {
    try {
        launch {
            val profile = async { fetchProfile() }
            val orders = async { fetchOrders() }
            println("profile: ${profile.await()} " +
                "orders: ${orders.await()}")
        }
    } catch (e: Exception) {
        println(e)
    }
    println("done")
}
```

On the default `runBlocking` event loop, `launch` schedules its child and returns a `Job`. The surrounding `try` finishes, then `done` prints. The `runBlocking` body reaches its end, but `runBlocking` itself cannot return while its child is still running.

```
runBlocking
+-- launch
    +-- async profile: waiting for 100 ms
    +-- async orders: throws after 50 ms

orders failure -> launch fails -> profile cancelled
               -> runBlocking fails -> main throws
```

Both `async` calls create children of `launch`. `Orders` fails with `IllegalStateException`, so the ordinary parent is cancelled and cancels `profile`. The failure propagates even though `orders.await()` has not run. `profile.await()` is interrupted by cancellation; the combined `println` never completes. The original cause eventually escapes `runBlocking`. [4, 11]

What “done” actually means here

It marks the end of the `runBlocking` block’s immediate statements, not successful completion of the whole coroutine tree.

Move a final message after `runBlocking` only if you want it to indicate successful completion of that call.

A catch inside `launch` around `await` alone is also not a reliable recovery boundary for normal `async` child failure: the parent `Job` may already be cancelled. Use a scoped function boundary for a failing group, or supervision plus explicit error handling for independent results.

Scope, context, and the Job tree

Three concepts that answer different questions

Concept	Question it answers
CoroutineScope	Where are new tasks launched, and who owns them?
CoroutineContext	Which keyed elements govern execution?
Job	What is this task's lifecycle and parent relationship?
Dispatcher	On which execution resource is work scheduled?
CoroutineName	How can this coroutine be identified in diagnostics?

```
fun main() = runBlocking(CoroutineName("lab")) {
    val parent = coroutineContext[Job]
    val child = launch(CoroutineName("profile")) {
        println(coroutineContext[Job])
        println(coroutineContext[CoroutineName])
    }
    println(parent)
    child.join()
}
```

Context is a set of elements indexed by keys. Combining contexts with + replaces an earlier element with the same key; it does not build a Job tree by adding several Job elements. A builder creates a new child Job whose parent comes from the launching scope. Other context elements, such as the dispatcher, are normally inherited or overridden. [16]

Jobs move from active toward completing/completed or cancelling/cancelled. Completion includes waiting for children; cancellation includes allowing cooperative cleanup. isCompleted includes both success and failure. isCancelled does not explain the failure policy by itself. [10]

Exercise 4

Draw the tree for runBlocking -> launch -> two async children. Then remove launch and draw it again. The async children now belong directly to runBlocking. Explain why changing a dispatcher does not change the ownership relationship.

Avoid passing a fresh Job to launch or withContext to “fix” failures. It can replace the inherited parent link and break ownership. Decide the lifecycle boundary explicitly before creating a long-lived scope.

Structured concurrency

A function should finish the work it promises to own

Use `coroutineScope` for a group that should succeed or fail as one operation. It inherits execution context, adds a scoped Job, waits for all children, and rethrows failure at its call boundary. That makes ordinary try/catch useful around the whole operation. [9]

```
suspend fun fetchAll(): Pair<String, String> = coroutineScope {
    val profile = async { fetchProfile() }
    val orders = async { fetchOrders() }
    profile.await() to orders.await()
}

fun main() = runBlocking {
    try {
        println(fetchAll())
    } catch (e: CancellationException) {
        throw e
    } catch (e: IllegalStateException) {
        println("Loading failed: ${e.message}")
    }
    println("done")
}
```

Orders failure cancels profile inside `fetchAll`. The group finishes its cleanup, then `fetchAll` throws to its caller. The caller handles the scoped operation's failure and remains usable: `done` prints. This is different from allowing a direct `async` child to fail the caller's own Job.

Ownership is a contract

A repository suspend function should normally return only when its requested work is complete, cancelled, or failed. Launching into a newly created unrelated scope and returning early changes that contract: the caller can no longer cancel or await the work through the function call.

Exercise 5

Add finally logging to both fetch functions. Run the failing group, then catch its exception. Verify that cleanup happens before the caller's final message. Next make orders succeed and compare sequential and concurrent timelines.

Some work intentionally outlives a screen, such as a durable background upload. Give it a named application owner or an appropriate Android background-work mechanism. Detaching a coroutine is not a durability guarantee; process death can still end it.

Official reading: [9] `coroutineScope`; [21] Android best practices. Sources are linked on pages 41-42.

Cancellation is cooperative

A cancellation request must be observed by the running work

Cancellation asks a Job to stop. Cancellable suspending operations such as `delay` and `await` observe it and throw `CancellationException`. A child cancelled this way does not normally fail its parent. Cancellation of a parent still travels down to children, including supervised children.

```
fun main() = runBlocking {
    val profile = launch {
        try {
            println(fetchProfile())
        } finally {
            println("profile cleanup")
        }
    }
    delay(20)
    profile.cancelAndJoin()
    println("parent continues")
}
```

`cancel()` requests cancellation; `join()` waits for termination; `cancelAndJoin()` combines them. After it returns, this child's finally block has completed. Cancelling a task does not roll back database writes or other side effects that already happened.

CPU loops need cancellation checks

```
suspend fun checksum(bytes: ByteArray): Long =
    withContext(Dispatchers.Default) {
        var total = 0L
        for (i in bytes.indices) {
            if (i % 4096 == 0) ensureActive()
            total += bytes[i].toLong()
        }
        total
    }
```

`ensureActive` throws when the current Job is cancelled. `yield` also gives other work a scheduling opportunity. A busy loop with neither checks nor suspension can delay cancellation indefinitely. Blocking code may need API-specific cancellation or `runInterruptible` when interruption is supported. [5, 6]

Exercise 6

Cancel profile after 20 ms and confirm that "profile" is absent but cleanup runs. For a CPU loop, compare cancellation with and without periodic `ensureActive`. Explain why putting a loop on `Default` does not itself make it cooperative.

Cleanup and deadlines

Make stopping safe without turning cancellation into a failure

```
suspend fun profileBeforeDeadline(): String? =
    withTimeoutOrNull(80) {
        fetchProfile()
    }
```

The deadline is shorter than the fixture's 100 ms delay, so this returns null. `withTimeout` instead throws `TimeoutCancellationException`. `withTimeoutOrNull` converts its own timeout into null; an outer cancellation still propagates. A timeout is cooperative, not a forced termination of arbitrary blocking code. [7]

Release resources in finally

```
suspend fun workWithCleanup() {
    try {
        delay(1_000)
    } finally {
        // Most cleanup should be immediate and non-suspending.
        withContext(NonCancellable) {
            delay(10) // Simulated bounded suspending cleanup.
            println("released")
        }
    }
}
```

`finally` executes during cancellation. Its normal suspending calls may immediately throw because the Job is cancelled. A small `NonCancellable` block allows necessary suspending cleanup to finish. Keep it bounded: a never-ending cleanup prevents the owning scope from finishing. Do not wrap normal fetching or long retries in `NonCancellable`.

When handling ordinary failures, catch `CancellationException` first and rethrow it. Broad `catch(Exception)` also matches cancellation; swallowing it can let cancelled work continue through non-suspending code and publish misleading results.

Exercise 7

Change the deadline to 150 ms and predict the value. Then cancel the outer caller while the deadline block runs. Confirm that outer cancellation is not reported as an orders error. Add a cleanup delay and explain when `NonCancellable` is necessary.

Close resources within the operation that owns them, using `use` or `try/finally`. Cancellation can race with returning a resource across a dispatcher or timeout boundary. Avoid assuming a closeable resource assigned outside a timed block will always be available for cleanup.

Official reading: [7] `withTimeout`; [12] `withContext`; [21] Best practices. Sources are linked on pages 41-42.

Exception propagation

Failure policy and error observation are separate mechanisms

Case	What to expect
Normal child launch fails	Parent fails and cancels siblings; error travels upward.
Normal child async fails	Deferred stores the error AND parent fails; await is not required for parent cancellation.
Supervised async fails	Siblings survive; await exposes the stored error.
Supervised launch fails	Siblings survive; unhandled exception still needs reporting or local handling.
Child cancelled	CancellationException normally does not fail its parent.

`CoroutineExceptionHandler` reports an uncaught exception after the coroutine has failed. It cannot restart that coroutine or return a fallback value. Normal children delegate failure to their parent; putting a handler on each child does not change the parent's failure policy. `async` exposes errors through `Deferred` instead of invoking its own handler. [4]

```
val handler = CoroutineExceptionHandler { _, error ->
    println("Uncaught: ${error.message}")
}
// Inside a runBlocking body:
supervisorScope {
    val child = launch(handler) {
        throw IllegalStateException("Wrong orders")
    }
    child.join() // Waits; does not retrieve/rethrow child failure.
    println("supervisor continues")
}
```

Use `try/catch` inside an operation when it can recover. Use a handler as last-resort reporting for suitable launch roots or supervised launch children. Use `await` and explicit handling for supervised `async` results. Supervision and exception reporting solve different problems.

Exercise 8

Predict whether a handler runs for supervised launch and supervised `async`. For `async`, add a caught `await`. For normal `async`, draw the upward cancellation path even if `await` is never called. Avoid using `GlobalScope` to hide the hierarchy.

Official reading: [4] Exceptions; [8] `supervisorScope`; [11] `async`. Sources are linked on pages 41-42.

coroutineScope vs supervisorScope

Choose whether a sibling's success remains useful after a failure

Use `coroutineScope` when all results are required. Use `supervisorScope` when independent child failures should not cancel siblings. Both functions wait for children, both remain attached to the caller, and caller cancellation cancels the whole group. [8, 9]

```
fun main() = runBlocking {
    supervisorScope {
        val profile = async { fetchProfile() }
        val orders = async { fetchOrders() }
        println("profile: ${profile.await()}")
        try {
            println("orders: ${orders.await()}")
        } catch (e: CancellationException) {
            throw e
        } catch (e: IllegalStateException) {
            println("orders unavailable: ${e.message}")
        }
    }
    println("done")
}
```

Expected output: profile: profile; orders unavailable: Wrong orders; done. Orders fails earlier, but its Deferred does not fail the supervisor. Profile finishes, then the stored orders failure is handled at await.

```
caller
+-- supervisorScope
    +-- profile async -> completes
    +-- orders async  -> fails (observed at await)

caller cancellation -> supervisor -> both children stop
```

The escaping-await trap

If `orders.await()` runs first and its error is not caught inside the supervisor block, the block itself throws. That fails the supervisor block and cancels its remaining children, including profile. Catching outside `supervisorScope` happens after that cancellation; it cannot preserve the unfinished child.

If profile can fail too, handle it independently. On page 20, both children produce explicit `Result` values. The requirement is a successful partial result, not merely “no crash.”

Job vs SupervisorJob

Scope functions are lexical; job objects need an owner

Job() creates an ordinary lifecycle parent. A non-cancellation child failure fails it and cancels siblings.

SupervisorJob() keeps direct children independent. Neither object supplies a dispatcher nor executes a block by itself. Build an owned CoroutineScope around it when a component needs a longer lifetime. [10, 13]

```
class ProfileWorker : AutoCloseable {
    private val owner = SupervisorJob()
    private val scope = CoroutineScope(
        owner + Dispatchers.Default + CoroutineName("worker")
    )
    fun refresh(): Job = scope.launch {
        try {
            println(fetchProfile())
        } catch (e: CancellationException) {
            throw e
        } catch (e: Exception) {
            println("Refresh failed: ${e.message}")
        }
    }
    override fun close() { owner.cancel() }
}
```

This illustrative component cancels its owned tasks when closed. close requests cancellation; it does not wait for cleanup. An owner requiring completed shutdown should expose a suspend stop method that cancels and joins. Android components usually use provided scopes instead of building another one.

Supervision applies at one parent-child edge

```
SupervisorJob
+-- launch A (ordinary coroutine Job)
|   +-- async profile
|   +-- async orders
+-- launch B
```

Orders failure still cancels profile because both are children of ordinary launch A. The supervisor protects B from A's failure; it does not automatically supervise grandchildren. Put supervisorScope inside A to make profile and orders independent there.

Exercise 9

Compare the tree above with two async calls launched directly in the supervisor scope. Explain why launch(SupervisorJob()) is a poor shortcut: it replaces ownership and the launch still has ordinary child-failure behavior.

Dispatchers and main safety

Move work according to what it does

Dispatcher	Use
Main	Android UI interaction and lightweight coordination.
Default	CPU-heavy parsing, sorting, image or data processing.
IO	Blocking file, socket, database, or other I/O calls.
Unconfined	Advanced execution behavior; not a default UI/background strategy.

A dispatcher controls scheduling, not success policy. Suspend APIs that already avoid blocking may not need an IO wrapper. Synchronous calls still block even inside a suspend function. Put dispatcher decisions in the class performing the blocking or CPU work so callers can safely invoke it from Main. Inject dispatchers for tests. [16, 21]

```
import java.io.File

class TextRepository(
    private val io: CoroutineDispatcher = Dispatchers.IO
) {
    suspend fun read(file: File): String = withContext(io) {
        file.readText() // Synchronous blocking I/O.
    }
}
```

The caller suspends while `read` executes on IO, then resumes in its original context. `withContext` does not guarantee a thread switch when the requested dispatcher is already the current one. It also does not make `File.readText` instantly cancellable midway through the call.

```
suspend fun pauseBlockingLibrary(): Unit =
    runInterruptible(Dispatchers.IO) {
        Thread.sleep(100) // This API supports interruption.
    }
```

Use `runInterruptible` for blocking JVM APIs that honor thread interruption. For APIs that do not, use their cancellation mechanisms or accept the bounded blocking behavior. Do not infer interruptibility merely from `withContext(IO)`. [6]

Exercise 10

Log thread names before, inside, and after `withContext`. Treat names as diagnostics rather than fixed assertions. Identify which operations in your real repository are blocking, CPU-heavy, or already suspending.

async vs withContext

Start concurrent values only when concurrency is useful

```
suspend fun sequentialValues(): Pair<String, String> {
    val p = withContext(Dispatchers.Default) { fetchProfile() }
    val o = withContext(Dispatchers.Default) {
        delay(50)
        "orders"
    }
    return p to o
}

suspend fun concurrentValues(): Pair<String, String> =
    coroutineScope {
        val p = async { fetchProfile() }
        val o = async { delay(50); "orders" }
        p.await() to o.await()
    }
```

The first function starts its orders work after profile completes: about 150 ms of simulated waiting. The second starts both before awaiting: about 100 ms. Awaiting p then o does not make those computations sequential; both were already started. [17]

withContext expresses “do this work in this context and return its result.” async expresses “start this child computation and give me a handle to a future value.” async(IO) followed immediately by await is usually less direct than withContext(IO) when there is no useful overlapping work.

Need	Choose
One result, move CPU or blocking work	withContext with an appropriate dispatcher
Two required independent computations	coroutineScope + async for both, then await
Two useful independent partial results	supervisorScope + explicit per-result error policy
Operation triggered by a UI action	launch in the owning UI/ViewModel scope

Exercise 11

First use successful orders so timing is understandable. Then restore its failure. Explain why replacing coroutineScope with supervisorScope changes failure isolation but does not fix an exception escaping the block.

Concurrency has costs: more requests in flight, more lifecycle edges, and more complicated error handling. Use it when overlap benefits the operation, not because every suspend call needs its own async.

Official reading: [11] async; [12] withContext; [17] Composition. Sources are linked on pages 41–42.

Model errors intentionally

Preserve successful values without erasing failure details

A nullable orders value cannot tell you whether the user has no orders or the request failed. `Result<T>` carries a value or a `Throwable`. A sealed domain type can instead distinguish success, offline, unauthorized, or unavailable. Select a model that lets the UI make the required decision.

```
suspend fun <T> attempt(
    block: suspend () -> T
): Result<T> = try {
    Result.success(block())
} catch (e: CancellationException) {
    throw e
} catch (e: Exception) {
    Result.failure(e)
}
```

This helper is a teaching boundary for recoverable `Exception` failures. Cancellation is rethrown and serious `Error` subclasses are not swallowed. In production, narrow expected failures when possible; a programming defect should not automatically become an ordinary “offline” UI state. [21, 24]

```
sealed interface OrdersOutcome {
    data class Ready(val value: String) : OrdersOutcome
    data object Unavailable : OrdersOutcome
}
// Map expected transport/service failures at the data boundary.
// Keep diagnostic causes in Result or logging when needed.
```

`runCatching` catches every `Throwable`, including `CancellationException`. It is convenient for synchronous computations, but using it unmodified around suspend work can turn cancellation into a normal `Result.failure`. It does not create supervision or repair an already cancelled Job. [24]

Exercise 12

Represent an empty successful orders list separately from a failed request. Add a retryable flag or domain error only if the caller uses it. Cancel attempt { `fetchProfile()` } and verify that cancellation escapes instead of returning a failure result.

A child that catches and converts an expected failure before it escapes can finish normally, even under `coroutineScope`. Supervision is most useful when child failure itself is allowed. Neither policy should consume the caller’s cancellation.

Complete partial-success loader

The profile survives an orders failure, with both outcomes retained

This complete main uses the fixtures on page 3 and attempt on page 19. Both children start before either await. Expected exceptions are modeled inside each computation, and the supervisor documents the independence of the two requests.

```
data class Dashboard(
    val profile: Result<String>,
    val orders: Result<String>
)

suspend fun loadDashboard(): Dashboard = supervisorScope {
    val profile = async { attempt { fetchProfile() } }
    val orders = async { attempt { fetchOrders() } }
    Dashboard(profile.await(), orders.await())
}

fun main() = runBlocking {
    val result = loadDashboard()
    println("profile: ${result.profile.getOrNull()}")
    result.orders.fold(
        onSuccess = { println("orders: $it") },
        onFailure = {
            println("orders unavailable: ${it.message}")
        }
    )
    println("done")
}
```

Expected output is profile: profile, orders unavailable: Wrong orders, then done. For these expected exceptions, attempt prevents failed child Jobs by turning failures into values. That means coroutineScope would also work in this particular implementation. supervisorScope additionally protects sibling work from an unmodeled child failure until that failure is observed or escapes the block.

Policy boundaries matter

If an unexpected exception escapes a child and later its await escapes the supervisor block, the operation fails and remaining children stop. Result modeling is not a promise to survive every possible defect. The owner can always cancel both requests.

Exercise 13: make the policy observable

Run four cases: both succeed, only orders fails, only profile fails, both fail. For each, inspect both Results. Add a fifth case that cancels the caller at 20 ms; neither result should be rendered as ordinary success or failure. Keep the public function attached to the caller.

Flow: a sequence over time

Use a stream when one final return value is not enough

A suspend function returns one result. `Flow<T>` can emit several values over time. The `flow { }` builder creates a cold stream: constructing it does not execute the producer; collecting it does. Each new collection runs that producer again. Some Flow implementations are hot; “Flow” alone does not imply cold. [18]

```
import kotlinx.coroutines.flow.*

fun profileUpdates(): Flow<String> = flow {
    emit("loading")
    emit(fetchProfile())
}

fun main() = runBlocking {
    val updates = profileUpdates() // No request yet.
    updates.collect { println("first: $it") }
    updates.collect { println("second: $it") }
}
```

Each collection prints loading then profile; `fetchProfile` runs twice. `collect` is a terminal suspending operation. Intermediate operators such as `map` build a pipeline without running it. A basic flow pipeline processes upstream and downstream sequentially in the collector’s coroutine unless an operator introduces a boundary.

```
fun normalizedProfiles(): Flow<String> =
    profileUpdates()
        .filter { it != "loading" }
        .map { it.uppercase() }
```

Cancelling the collecting coroutine stops this suspending producer. `emit` in flow is cancellation-aware. A CPU-heavy producer still needs cooperative work. Do not emit from a different dispatcher using `withContext` inside flow; use `flowOn` for upstream context, or a suitable builder such as `channelFlow` for concurrent producers.

Exercise 14

Add a “fetch started” log. Create the Flow without collecting and count logs; then collect twice. Predict the number of requests. Decide whether your UI needs a fresh request for each subscriber or a shared source.

Official reading: [18] flow builder; [19] flowOn. Sources are linked on pages 41-42.

Flow operators and errors

Place transformations and recovery at deliberate boundaries

```
fun profileStream(): Flow<Result<String>> = flow {
    emit(Result.success(fetchProfile()))
}.catch { e ->
    if (e is CancellationException) throw e
    emit(Result.failure(e))
}
```

catch handles exceptions in the upstream pipeline. It does not handle exceptions thrown by the downstream collector or cancellation used to stop the stream. Moving catch changes which work it protects. Put UI rendering failures where they can be diagnosed instead of pretending they are network failures. [20]

```
fun labels(): Flow<String> = flow {
    emit(fetchProfile())
}.map { it.uppercase() }
    .flowOn(Dispatchers.Default)
```

flowOn changes upstream execution context and leaves downstream collection in the caller's context. Do not put a Job into flowOn's context. Cross-dispatcher boundaries can buffer values; cancellation may mean an in-flight value is never delivered. [19]

Operator	Practical meaning
buffer	Lets producer and consumer overlap with a queue.
conflate	A slow consumer can skip intermediate values and see newer ones.
collectLatest	A new value cancels the previous collector action.
combine	After all sources emit, uses their latest values on updates.
zip	Pairs corresponding emissions from two sources.
retry / retryWhen	Recollect upstream on selected failures; use bounds and backoff.

Exercise 15

Emit versions every 20 ms while rendering takes 80 ms. Compare collect, conflate, and collectLatest. Do not assert every intermediate value is seen in the latter two. For combine, delay the first orders emission and predict when a combined dashboard becomes available.

Retries restart upstream and can repeat side effects. Use bounded retries for transient failures and check idempotency. Model partial errors before combining sources if one failure must not terminate the dashboard. Operator reading: [27].

StateFlow: the current state

A hot, observable value with an initial state

StateFlow always has a value and immediately gives a new collector its current state. It suppresses equal updates using equality and conflates changes: slow collectors may miss intermediate states. It does not complete normally or encode failure automatically. Represent loading and errors in your state type. [14]

```
data class ScreenState(
    val loading: Boolean = false,
    val profile: String? = null,
    val orders: String? = null,
    val ordersError: String? = null,
    val profileError: String? = null,
    val generalError: String? = null
)
val mutableState = MutableStateFlow(ScreenState())
val state: StateFlow<ScreenState> = mutableState.asStateFlow()

fun showLoading() {
    mutableState.update { it.copy(loading = true) }
}
```

Expose the read-only StateFlow and keep mutation private. update performs an atomic transformation; its lambda may be evaluated more than once under contention, so avoid side effects inside it. Use immutable copies rather than mutating a list already stored in the state, which can hide a meaningful change behind equality.

```
// Inside a lifecycle-owned component with a scope:
val sharedProfiles: StateFlow<String> = profileUpdates().stateIn(
    scope = scope,
    started = SharingStarted.WhileSubscribed(5_000),
    initialValue = "loading"
)
```

stateIn shares a cold upstream as state. The scope owns that sharing coroutine. WhileSubscribed starts upstream when subscribers appear and can stop it after the timeout. The timeout is a deliberate lifecycle policy, not a universal constant. Handle upstream failures before stateIn if the UI should receive an error state. [15]

Exercise 16

Set the same state twice and observe equality suppression. Publish several states quickly and explain why a slow collector is allowed to skip some. Test the final state value when your requirement is current UI state, rather than forcing every intermediate emission.

SharedFlow: broadcast values

Configure replay and buffering around delivery requirements

SharedFlow broadcasts to active collectors and can replay recent values to new collectors. It has no required initial value. It never completes normally, so collectors usually need cancellation or operators such as `take`. Replay, buffer capacity, and overflow policy determine what can be retained or dropped. [23]

```
sealed interface UiEvent {
    data class Message(val text: String) : UiEvent
}
private val mutableEvents = MutableSharedFlow<UiEvent>(
    replay = 0,
    extraBufferCapacity = 1
)
val events: SharedFlow<UiEvent> = mutableEvents.asSharedFlow()

suspend fun notifyOrdersUnavailable() {
    mutableEvents.emit(UiEvent.Message("Orders unavailable"))
}
```

This is an ephemeral notification policy. With `replay = 0`, an event emitted when there are no subscribers is not kept for a future subscriber; `extraBufferCapacity` does not provide offline storage. `tryEmit` returning `true` is not proof that a UI collector received the event. With `replay > 0`, a recreated collector may receive an old event again.

Type	Best fit	Important limit
Cold Flow	A request or computation per collection	Two collectors may repeat the work.
StateFlow	The latest durable-in-memory UI state	Intermediate values may be skipped.
SharedFlow	Broadcast updates or ephemeral signals	Delivery depends on subscribers and configuration.

If a payment result or essential navigation decision must survive a collector absence, store a pending state or durable record with acknowledgment rather than assuming a broadcast gives exactly-once delivery. None of these types survives process death by itself.

Exercise 17

Emit one event before subscribing and one after. Repeat with `replay = 1`. Explain both outcomes and choose a policy for an optional snackbar versus an essential confirmation. Cancel your collectors when the exercise ends.

Official reading: [23] SharedFlow; [15] Android state and sharing. Sources are linked on pages 41-42.

Android: a ViewModel owns loading

Expose state; keep the request attached to the screen model

viewModelScope is cancelled when its ViewModel is cleared. Its top-level children are supervised, but children inside one launch still have that launch as an ordinary parent. Our loadDashboard function supplies the independent-request policy within that launch. [25]

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

class DashboardViewModel : ViewModel() {
    private val mutableState = MutableStateFlow(ScreenState())
    val state = mutableState.asStateFlow()
    private var refreshJob: Job? = null
    fun refresh() { // Call from the UI/Main thread.
        refreshJob?.cancel() // Latest refresh wins.
        refreshJob = viewModelScope.launch {
            mutableState.update { it.copy(loading = true) }
            try {
                val result = loadDashboard()
                mutableState.value = ScreenState(
                    profile = result.profile.getOrNull(),
                    orders = result.orders.getOrNull(),
                    ordersError = result.orders.exceptionOrNull()
                        ?.message,
                    profileError = result.profile.exceptionOrNull()
                        ?.message
                )
            } catch (e: CancellationException) {
                throw e
            } catch (e: Exception) {
                mutableState.update {
                    it.copy(loading = false, generalError = e.message)
                }
            }
        }
    }
}
```

This uses ScreenState (23) and loadDashboard (20). Cancellation prevents publishing a cancelled result. Repository calls must be main-safe; viewModelScope does not move blocking code off Main. For production, map diagnostic exception messages to stable user-facing text.

Exercise 18

Inject a repository instead of top-level fixtures. Trigger refresh twice and verify only the newest result is published. Add a deliberate policy for retaining stale profile data during a failed refresh.

Android: collect with the lifecycle

Ownership of loading and ownership of rendering are different

A Fragment's view can be destroyed while the Fragment object remains. Collect using `viewLifecycleOwner` so rendering stops with the view. `repeatOnLifecycle` starts its block at `STARTED`, cancels it below `STARTED`, and restarts it on reentry. It does not automatically cancel separate `ViewModel` requests. [25]

```
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.lifecycleScope
import androidx.lifecycle.repeatOnLifecycle

// Inside Fragment.onViewCreated; viewModel and render exist.
viewLifecycleOwner.lifecycleScope.launch {
    viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
        viewModel.state.collect { state -> render(state) }
    }
}
```

For two endless flows, collect each in a child launch within `repeatOnLifecycle`. Sequential collect calls never reach the second call while the first stream is still active. If either collector fails, the ordinary repeat block's children fail as a group unless you intentionally add a different policy.

```
import androidx.compose.runtime.getValue
import androidx.lifecycle.compose.collectAsStateWithLifecycle

// Inside a @Composable screen:
val state by viewModel.state.collectAsStateWithLifecycle()
// Render widgets from state.
```

Compose collection turns `StateFlow` into lifecycle-aware Compose state. Keep rendering cheap and move requests to explicit `ViewModel` actions or properly keyed effects. Recomposition alone should not start a fresh network request.

Project dependencies

Use matching, compatible AndroidX Lifecycle artifacts: `lifecycle-viewmodel-ktx` and `lifecycle-runtime-ktx` for the `ViewModel`/`Fragment` examples, and `lifecycle-runtime-compose` for Compose collection. Add `kotlinx-coroutines-android` matching your core version to provide `Main` dispatching. Use your project's version catalog and current Android documentation for AndroidX versions. [15, 25]

Exercise 19

Navigate away and back; rotate the device; then remove the screen. Log collection starts/stops and request starts/stops separately. Explain which lifecycle owns each. For persistent work across process death, study `WorkManager` rather than relying on a coroutine scope.

Coroutine tests: virtual time

Control scheduling instead of sleeping and hoping

`runTest` supplies a `TestScope` and scheduler, skips delays on test dispatchers, and reports uncaught child failures. A `StandardTestDispatcher` queues work; `runCurrent` runs ready work, `advanceTimeBy` moves the virtual clock, and `advanceUntilIdle` drains scheduled work. Share one scheduler across test dispatchers. [22, 26]

```
import kotlinx.coroutines.*
import kotlinx.coroutines.test.*
import kotlin.test.*

@OptIn(ExperimentalCoroutinesApi::class)
class ProfileTest {
    @Test fun profileUsesVirtualTime() = runTest {
        val profile = async { fetchProfile() }
        runCurrent() // Starts the async body at virtual t = 0.
        assertFalse(profile.isCompleted)
        advanceTimeBy(100)
        runCurrent() // Executes tasks scheduled exactly at t = 100.
        assertEquals("profile", profile.await())
        assertEquals(100L, testScheduler.currentTime)
    }
}
```

The test verifies the fixture's result and timing contract without waiting 100 real milliseconds. Tasks exactly at the target time may require `runCurrent` after `advanceTimeBy`. For outcome tests, prefer `advanceUntilIdle` or directly awaiting when exact timing is not part of the requirement.

Make dispatchers controllable

Inject a `TestDispatcher` into repositories rather than hard-coding IO or Default. Delays on real dispatchers do not use this virtual clock. For ViewModel JVM tests, replace `Dispatchers.Main` with a test dispatcher before creating the ViewModel, and reset Main afterward, usually with a reusable test rule. [26]

Exercise 20

Change the fixture delay to 120 ms and update only a test that deliberately checks that timing. Write another test that cares only about the returned profile. Explain why a thread name assertion is less robust than a result assertion.

For endlessly running test collectors, launch in `backgroundScope` so `runTest` can cancel them at the end, or explicitly `cancelAndJoin` them. An infinite loop can prevent `advanceUntilIdle` from reaching idle.

Test the failure policy

Verify partial success, grouped failure, and caller cancellation

```
@OptIn(ExperimentalCoroutinesApi::class)
class DashboardTest {
    @Test fun ordersFailureKeepsProfile() = runTest {
        val dashboard = loadDashboard()
        assertEquals("profile", dashboard.profile.getOrThrow())
        assertIs<IllegalStateException>(
            dashboard.orders.exceptionOrNull()
        )
    }
    @Test fun cancellationIsNotAResultFailure() = runTest {
        var returned = false
        var cleaned = false
        val job = launch {
            try {
                loadDashboard()
                returned = true
            } finally {
                cleaned = true
            }
        }
        runCurrent()
        advanceTimeBy(20)
        job.cancelAndJoin()
        assertFalse(returned)
        assertTrue(cleaned)
        assertTrue(job.isCancelled)
    }
}
```

These tests use the imports on page 27. The first proves the desired partial-success behavior. The second proves cancellation escapes the loader and cleanup finishes; it does not merely assert that an `Exception` was recorded.

Test state and streams by their contracts

For a finite cold `Flow`, collect with `toList` and assert the ordered sequence. For `StateFlow`, assert the latest value when that is the requirement; conflation means every rapid intermediate update need not appear. When testing `stateIn` with `WhileSubscribed`, start an active collector or upstream will not run. For `SharedFlow` `replay = 0`, arrange for a collector to start before emission. [14, 15, 26]

Exercise 21

Add an all-or-nothing test for `fetchAll`. Track a finally flag in profile and assert cleanup after orders fails. Then make orders succeed and verify both values. Extend the cancellation test to the CPU checksum and to a lifecycle-owned refresh.

Verify policy and lifetime with controlled dispatchers. Avoid real sleeps and wall-clock assertions. Check Android navigation and configuration changes on a device or emulator too.

Five-week schedule / weeks 1-2

45-60 minutes each day; finish with a runnable experiment

Day	Learn and do	Proof of understanding
1	Setup and functions (3-4). Run the first main.	Explain unresolved delay and dependency/import roles.
2	Types, nullability, exceptions (4). Exercise 1.	Predict try/catch/finally output without running.
3	Lambdas and receivers (5). Exercise 2; snippet placement (35).	Explain why launch resolves in a scope.
4	Threads and blocking (6). Exercise 3.	Draw blocking vs suspending lanes.
5	Suspend functions (7). Sequential fixture.	Explain why suspend does not start a task.
6	Review checkpoint A; repeat weakest example.	Give a 3-minute explanation with no notes.
7	Rest or light review; annotate your lab notebook.	List three predictions you initially got wrong.
8	Builders (8). Compare Job and Deferred.	Explain join vs await and their cancellation.
9	Original program (9). Reproduce the failure.	Explain done, missing profile, and escaped error.
10	Scope/context/Job (10). Exercise 4.	Draw and label the complete ownership tree.
11	coroutineScope (11). Exercise 5.	Catch the grouped failure at its call boundary.
12	Make orders succeed; compare sequential/concurrent.	Start both async calls before either await.
13	Complete lab A on page 32.	Predict outcomes after moving/removing launch.
14	Review checkpoint B on page 33.	Explain scope ownership without thread jargon.

Keep a notebook entry for each session: prediction, observed output, Job tree, explanation, and one changed variable. Timing experiments should use successful fixtures first; adding exceptions before understanding overlap obscures the concurrency lesson.

Adapt the pace

If syntax is already familiar, spend the saved sessions on exception experiments. Do not compress the supervision or cancellation sections just because their APIs are short.

Five-week schedule / weeks 3-4

Turn the execution model into deliberate application behavior

Day	Learn and do	Proof of understanding
15	Cancellation and CPU loops (12). Exercise 6.	Cancel a child without failing its parent.
16	Cleanup and deadlines (13). Exercise 7.	Distinguish timeout from outer cancellation.
17	Exception propagation (14). Exercise 8.	Predict normal vs supervised async failure.
18	Supervision and owned Jobs (15-16). Exercise 9.	Explain the direct-child supervision limit.
19	Dispatchers (17-18), shared-state lab (38).	Place blocking work and choose async only for overlap.
20	Result and partial loader (19-20, 36). Exercises 12-13.	Preserve partial success and rethrow cancellation.
21	Lab B (32) and checkpoint C (33).	Run every failure combination and cancellation.
22	Cold Flow (21). Exercise 14; worked stream test (37).	Predict work performed by two collectors.
23	Operators and error boundaries (22). Exercise 15.	Explain catch placement and collectLatest cancellation.
24	StateFlow (23). Exercise 16.	Model loading, success, and errors explicitly.
25	SharedFlow (24). Exercise 17.	Demonstrate no-subscriber loss with replay = 0.
26	Android ViewModel (25). Exercise 18.	Implement latest-refresh-wins with main-safe data calls.
27	Lifecycle collection (26). Exercise 19.	Trace collection and loading lifetimes separately.
28	Review checkpoint D; practice on device/emulator.	Observe stop/restart and rotation behavior.

When translating console examples to Android, replace the demo `runBlocking` entry point with `viewModelScope.launch`. Keep the suspend loader reusable. Avoid rewriting the failure policy separately in every screen.

Decision checkpoint

Before adding a stream or event channel, state what must survive: a sibling failure, a collector pause, view recreation, or process death. These are four different lifetime requirements.

Five-week schedule / week 5

Test, consolidate, and build a small dashboard capstone

Day	Learn and do	Deliverable
29	Test setup (35), virtual time (27). Exercise 20.	A passing fixture test without real sleeps.
30	Failure policy tests (28). Exercise 21.	Partial success, grouped failure, cancellation tests.
31	Lab C streams (32). Add lifecycle verification.	Flow, StateFlow, and SharedFlow observations.
32	Capstone: repository and immutable UI state.	Both requests run concurrently; errors modeled.
33	Capstone: ViewModel, refresh, lifecycle collector.	Latest refresh wins; rendering is lifecycle-aware.
34	Capstone: Android tests (39), review rubric (40).	A compact evidence log for behavior and lifetime.
35	Final checkpoint (33), teach-back, clean up.	Explain the original program and your final solution.

Capstone acceptance criteria

Build a profile/orders dashboard backed by fake repositories first. Start both requests concurrently. Show a successful profile when orders fails. Represent empty orders separately from unavailable orders. Cancel stale refreshes, keep repository work main-safe, and expose immutable StateFlow to the UI.

Add tests for both success, either failure, both failures, and caller cancellation. Verify that essential state remains available after lifecycle collection restarts. If you add snackbar events, demonstrate your chosen replay/drop behavior rather than claiming guaranteed delivery.

What a good final explanation includes

“The caller owns the loading operation. Its supervisor controls direct-child failure isolation. Each request maps expected errors to a Result and preserves CancellationException. The ViewModel owns refresh lifetime; the view lifecycle owns rendering. Dispatchers describe execution resources. Tests replace those resources and advance virtual time.”

Graduation criterion

Without looking at the guide, draw the original Job tree, predict its output, fix all-or-nothing handling, fix partial-success handling, and explain why changing only await order or only the dispatcher cannot accomplish those policies.

Progressive labs

Use a small controlled change to expose each rule

Lab A / execution and ownership

1. Use successful profile (100 ms) and successful orders (50 ms). Implement sequential calls, then two async calls started before awaiting. Predict approximately 150 ms vs 100 ms. Explain overlap without assuming two threads.
2. Restore failing orders. Run the original launch program. Predict done before the failure, no completed profile print, and `IllegalStateException` escaping `runBlocking`. Remove launch and redraw the tree. Failure still reaches the ordinary parent.
3. Wrap the group in `fetchAll` using `coroutineScope`. Catch the call's failure in its caller. Confirm profile cancellation and caller continuation. Add finally flags rather than relying only on console timestamps.

Lab B / independent results and cancellation

1. Use `supervisorScope` with two async calls. Await orders first without a local catch. Predict profile cancellation because the supervisor block throws. Now handle the orders exception inside that block and prove profile finishes.
2. Use `attempt` and `loadDashboard`. Execute the four success/failure combinations. Preserve the original exception causes in the Results. Cancel the caller at 20 ms and assert that the loader does not return a Dashboard.
3. Introduce a nested launch inside a supervisor and place both async calls inside that launch. Predict profile cancellation on an unhandled orders error. Move supervision to their direct parent and retest.

Lab C / streams and Android lifetime

1. Collect `profileUpdates` twice and count request starts. Convert a cold source to `stateIn` using a named owner and an explicit sharing policy. Observe whether subscriber absence stops or restarts upstream.
2. Emit a `SharedFlow` event with no collector. Show the difference between `replay = 0` and `replay = 1`. Choose which events can be lost and which must be state or durable records.
3. Build the Android dashboard. Rotate, background/foreground, and navigate away. Log the `ViewModel` lifetime, request lifetime, and view collection lifetime separately. Add virtual-time tests for the loader and refresh policy.

Stretch experiment

Make profile take 50 ms and orders fail at 100 ms. Under ordinary structured concurrency, profile may finish before failure; the group still fails. A completed side effect is not rolled back. Explain how this differs from the original 100/50 ms fixtures.

Checkpoints with answer cues

Try each question before reading its answer

A / syntax and execution

Does suspend mean background? No. It permits suspension; the current context governs execution. Is delay blocking? It suspends the coroutine. Why does launch resolve here? Its CoroutineScope receiver is supplied by the enclosing builder block.

B / ownership and the original failure

Why does done print early? It belongs to the immediate block statements, while runBlocking still waits for children. Must orders.await run for parent cancellation? No. A normal async child's failure already fails its parent. Does join retrieve a child exception? No; it waits for completion and may itself be cancelled with its waiter.

C / cancellation and supervision

Does catching await repair a cancelled ordinary parent? No. A Job's cancellation is not reversible. Does supervision prevent caller cancellation? No; cancellation still travels down. What if await throws out of supervisorScope? The block fails and cancels unfinished children. Does SupervisorJob supervise grandchildren? Only its direct-child edges; an ordinary intervening launch still groups its children.

Should Result catch cancellation? No; propagate CancellationException. Does withContext(IO) interrupt any blocking API? No. Use cancellation-aware APIs or interruption where supported. When use coroutineScope? When one result is useless without the other and the operation should fail as a group.

D / streams and Android

Do two cold collectors share one request? Normally no; each executes upstream. Does StateFlow deliver every state transition? No; it conflates and suppresses equal updates. Does extraBufferCapacity store events with no subscriber? With replay = 0, no. What survives a view collection stopping? ViewModel-owned work can continue; rendering collection restarts according to lifecycle policy.

E / testing and final teach-back

Why inject dispatchers? To control scheduling and keep virtual-time tests deterministic. Why runCurrent after advancing to 100 ms? To execute tasks queued exactly at that boundary. How test caller cancellation? Cancel and join the calling Job; assert cleanup and that no normal result was published.

Score your understanding

Answer the related checkpoint questions correctly and explain one modified example from scratch before moving on. API recall without a correct Job tree does not count as a completed checkpoint.

Quick reference and glossary

A compact decision sheet to keep beside your editor

If you need to...	Use or remember...
Start owned work without a returned value	launch; catch expected failures inside the operation.
Compute concurrent results	async in an owned scope; start both, then await.
Move one blocking/CPU operation	withContext(IO/Default); inject dispatchers.
Fail a request group together	coroutineScope; catch at its suspend call boundary.
Keep independent sibling requests alive	supervisorScope; handle failures inside its block.
Create a component-owned scope	Job or SupervisorJob plus dispatcher; define shutdown.
Stop work and wait for cleanup	cancelAndJoin; work must cooperate.
Represent recoverable errors	Result or domain types; rethrow cancellation.
Produce multiple values	Flow; check cold vs hot ownership.
Expose current UI state	Read-only StateFlow; immutable state updates.
Broadcast ephemeral values	SharedFlow with intentional replay/buffer policy.
Render Android state safely	repeatOnLifecycle / collectAsStateWithLifecycle.
Test coroutine policy	runTest; one scheduler; controlled dispatchers.

Words to know

Continuation: saved next step for resumable computation. Scope: launching boundary with an execution context. Context: keyed execution and lifecycle elements. Job: task lifecycle and ownership handle. Deferred: Job with a result. Structured concurrency: child work remains attached to an owner that waits for it. Supervision: independent failure across direct children. Conflation: skipping intermediate updates in favor of newer values. Virtual time: a controlled test clock for scheduled work.

Three recurring mistakes

A catch around launch does not catch its later body failure. async does not delay normal parent failure until await. supervisorScope does not make an exception escaping its block harmless.

Run snippets and discover tests

Remove setup friction before studying behavior

A .kt file accepts declarations such as functions and classes at file scope, but not loose statements such as `println` or `attempts += 1`. Place those inside a function. The syntax listings on pages 4-5 mix declarations and usage to explain them; this is how to run that usage:

```
fun label(id: Int): String = "user-$id"
fun main() {
    val profile = label(7)
    var attempts = 0
    attempts += 1
    println("profile: $profile, attempts: $attempts")
}
```

Use `src/main/kotlin/Main.kt` for the demo, and `src/test/kotlin/ProfileTest.kt` for the class on page 27. Keep your generated Kotlin/JVM plugin and compatible JDK configuration. The setup below uses Jupiter via the Kotlin JUnit integration; JUnit 6 requires Java 17 or later. [31]

```
// Add to build.gradle.kts alongside the core/test dependencies.
dependencies {
    testImplementation(kotlin("test-junit5"))
    testImplementation(platform("org.junit:junit-bom:6.0.3"))
    testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
}
tasks.test { useJUnitPlatform() }
```

Sync Gradle, then run the test using its IDE gutter icon or `./gradlew test` from the project folder. A passing test run must report tests executed, not “no tests found.” Temporarily change the expected profile value and verify a failure; restore the correct assertion afterward.

Keep the configurations distinct

The Android host project can use its existing JUnit configuration. Do not copy this standalone JVM setup into an Android module without adapting it. `kotlin.test` assertions work with the selected Kotlin/JUnit integration.

Use one main per experiment. Keep fixtures and reusable helpers in the same package as tests, or import them. The two-argument loader on page 36 is an overload of the zero-argument loader on page 20, so both can coexist.

Worked solution: the failure matrix

Inject behavior to test all outcomes without editing fixtures

Keep Dashboard (20) and attempt (19). This overload accepts suspend function values instead of fixed network fixtures. It remains attached to its caller and preserves the same independent-result contract.

```
suspend fun loadDashboard(
    profileRequest: suspend () -> String,
    ordersRequest: suspend () -> String
): Dashboard = supervisorScope {
    val p = async { attempt { profileRequest() } }
    val o = async { attempt { ordersRequest() } }
    Dashboard(p.await(), o.await())
}
```

Put this test in `src/test/kotlin`. Use the imports on page 27. Each case takes 100 virtual ms, demonstrating that both delayed requests start before either is awaited. Success or failure changes the result, not whether both tasks are attempted.

```
@OptIn(ExperimentalCoroutinesApi::class)
class FailureMatrixTest {
    @Test fun checksEveryCombination() = runTest {
        for (failP in listOf(false, true)) {
            for (failO in listOf(false, true)) {
                val started = testScheduler.currentTime
                val result = loadDashboard(
                    profileRequest = {
                        delay(100)
                        check(!failP) { "Wrong profile" }
                        "profile"
                    },
                    ordersRequest = {
                        delay(50)
                        check(!failO) { "Wrong orders" }
                        "orders"
                    }
                )
                assertEquals(!failP, result.profile.isSuccess)
                assertEquals(!failO, result.orders.isSuccess)
                assertEquals(100L, testScheduler.currentTime - started)
            }
        }
    }
}
```

Expected cases: success/success, success/failure, failure/success, failure/failure. Cancellation is a separate fifth case, covered on page 28; it must abort the call rather than return two ordinary failed Results. A failure converted inside attempt does not fail its child Job.

Worked solution: streams

Observe combined state and prove what broadcasts retain

combine waits for both sources to emit and then updates using their latest values. In this finite example, orders supplies loading at $t = 0$; profile arrives at $t = 100$; orders becomes ready at $t = 150$. The expected combined list contains two states. [27]

```
class StreamTest {
    @Test fun combinesLatestValues() = runTest {
        val profile = flow { emit(fetchProfile()) }
        val orders = flow {
            emit("loading")
            delay(150)
            emit("orders")
        }
        val values = profile.combine(orders) { p, o -> p to o }
            .toList()
        assertEquals(listOf("profile" to "loading",
            "profile" to "orders"), values)
    }
}
```

toList waits for completion. On endless StateFlow or SharedFlow, use take or a cancellable collector. Below, take(1) finishes after the first event. [23, 26]

```
@OptIn(ExperimentalCoroutinesApi::class)
class BroadcastTest {
    @Test fun noSubscriberMeansNoReplay() = runTest {
        val events = MutableSharedFlow<String>(replay = 0,
            extraBufferCapacity = 1)
        events.emit("before")
        val received = mutableListOf<String>()
        backgroundScope.launch {
            events.take(1).toList(received)
        }
        runCurrent() // Subscribe before the next emission.
        events.emit("after")
        runCurrent()
        assertEquals(listOf("after"), received)
    }
}
```

Try one controlled change

Set `replay = 1`. The first received value becomes before, because the new subscriber receives replayed history. This is a different delivery contract, not a guarantee of exactly-once handling. Use the Flow and test imports on pages 21 and 27.

Worked solution: shared state

Even one thread can lose updates across suspension

“Read a value, suspend, write a changed value” is not one indivisible operation. Another coroutine can read the same old value during the suspension. Dispatchers decide execution resources; they do not make this sequence atomic. [30]

```
import kotlinx.coroutines.sync.Mutex
import kotlinx.coroutines.sync.withLock

suspend fun countUpdates(protected: Boolean): Int = coroutineScope {
    var count = 0
    val mutex = Mutex()
    val workers = List(100) {
        launch {
            suspend fun increment() {
                val before = count
                yield() // Deliberately exposes interleaving.
                count = before + 1
            }
            if (protected) mutex.withLock { increment() }
            else increment()
        }
    }
    workers.joinAll() // Read the result only after work finishes.
    count
}
```

joinAll before reading count matters: returning count and letting coroutineScope wait afterward would capture the old Int value. Waiting for children does not recompute the block's result.

```
class CounterTest {
    @Test fun protectsTheWholeUpdate() = runTest {
        assertTrue(countUpdates(protected = false) < 100)
        assertEquals(100, countUpdates(protected = true))
    }
}
```

The StandardTestDispatcher interleaves the unprotected reads at yield. Mutex protects the entire read/write sequence; waiting for its lock suspends rather than blocks a thread. It is non-reentrant: trying to lock the same mutex again from within its held block can deadlock. Keep real critical sections small. [32]

Choose protection by the invariant

An atomic increment suffices for a single counter. A Mutex can protect several related values as one invariant.

StateFlow.update atomically transforms one state value; it does not protect a network request plus another variable.

Never put side effects in an update lambda that may be retried.

Worked solution: Android unit tests

Install Main before the ViewModel is constructed

Local JVM tests have no Android UI thread. Replace Main with a test dispatcher before using `viewModelScope`; reset it in finally. This test uses the ViewModel from page 25, its fixtures and `ScreenState`, plus imports from pages 25 and 27. It needs the host project's compatible AndroidX dependencies. [26]

```
@OptIn(ExperimentalCoroutinesApi::class)
class DashboardViewModelTest {
    @Test fun publishesThePartialResult() = runTest {
        Dispatchers.setMain(StandardTestDispatcher(testScheduler))
        var model: DashboardViewModel? = null
        try {
            val vm = DashboardViewModel()
            model = vm
            vm.refresh()
            runCurrent()
            assertTrue(vm.state.value.loading)
            advanceUntilIdle()
            assertFalse(vm.state.value.loading)
            assertEquals("profile", vm.state.value.profile)
            assertEquals("Wrong orders", vm.state.value.ordersError)
        } finally {
            model?.viewModelScope?.cancel()
            Dispatchers.resetMain()
        }
    }
}
```

`setMain` affects process-wide dispatcher state, so do not run tests that replace it concurrently. Creating the test dispatcher from `testScheduler` explicitly ensures the ViewModel and `runTest` use the same virtual clock. Cancelling its scope prevents this test from leaving owned work running; it is not an assertion about the framework clearing the ViewModel.

Configuration change is not ViewModel clearing

A correctly scoped ViewModel usually survives a configuration change such as rotation. `viewModelScope` is cancelled when that owner actually clears it, not merely whenever a view is destroyed. Navigation behavior depends on the ViewModel's Activity, Fragment, or navigation-entry owner. [25]

Verify on a device or emulator

Start a refresh and rotate: the old view collector stops, the new collector observes retained state, and the same ViewModel need not refetch. Navigate so its owner is removed: verify clearing and cancellation. Background the app: `repeatOnLifecycle` stops rendering collection; ViewModel work can continue while the process lives.

Confine refresh calls to Main before accessing `refreshJob`. With injected repositories, test slow-old/fast-new requests: only the latest should publish completed state.

Readiness review and next steps

What completing this guide should and should not establish

This guide is sufficient for the original goal: understanding the profile/orders program from Kotlin syntax through ownership, cancellation, exceptions, supervision, Android state, and tests. Completing the labs is part of that goal; reading the explanations alone does not demonstrate the behavior.

A final assessment with concrete evidence

Skill	Evidence you should produce
Trace ownership	Draw each Job edge in the original and supervised programs.
Explain failure	Predict output before running; explain why await order cannot defer normal child failure.
Choose a result policy	Implement both all-or-nothing and partial-success loaders.
Preserve cancellation	Cancel at 20 ms; assert no normal Dashboard return and completed cleanup.
Handle state	Distinguish empty data from failed data and define stale-data behavior.
Handle stream lifetime	Demonstrate cold recollection, StateFlow conflation, and no-subscriber event loss.
Test Android integration	Replace Main in a JVM test; observe view collection and ViewModel lifetime on a device.

Topics to study after this foundation

Learn callback bridging with `suspendCancellableCoroutine` and `callbackFlow` when integrating listener APIs. Study Channels and bounded concurrency when tasks need queues or many requests. Study `WorkManager` and persisted state when work must survive process death. These are follow-on topics rather than prerequisites for your original two-request example.

Review record

The review checked the core rules against official Kotlin and Android documentation, corrected prose formatting of generic types, clarified runnable snippet placement, added test-engine setup, and supplied worked failure, stream, synchronization, and Android test examples.

Verification ran the central JVM examples, failure matrix, stream and synchronization checks, and the printed ViewModel unit test. Test bodies were compiled and invoked with assertions; Gradle/JUnit discovery is a separate setup check on page 35. Device lifecycle, navigation, and process-death behavior still require your Android project.

When to move on

Move on when you can explain a changed example from its ownership tree and verify it with a controlled test. Repeat the lab if your explanation depends only on print order or thread names.

Official reading / foundations and ownership

Primary references, linked for deeper study

[1] [Coroutines basics](#)

Start here for setup, builders, suspension, and structured concurrency.

[2] [Higher-order functions and lambdas](#)

Read function types and function literals with receivers to decode builder syntax.

[3] [Exception and error handling](#)

Review try as an expression, catch types, finally, and throwing.

[4] [Coroutine exceptions handling](#)

Read exception propagation and supervision after drawing the original Job tree.

[5] [ensureActive API](#)

Cancellation checks for long-running nonsuspending work.

[6] [runInterruptible API](#)

Bridge JVM blocking APIs that support interruption.

[7] [withTimeout API](#)

Understand cooperative timeout and cancellation/resource races.

[8] [supervisorScope API](#)

Study block failure, sibling isolation, and unhandled launch exceptions.

[9] [coroutineScope API](#)

Review lexical ownership and the exception boundary.

[10] [Job API](#)

Lifecycle states, parent-child semantics, cancellation, and join.

[11] [async API](#)

Result storage, default parent cancellation, and start modes.

[12] [withContext API](#)

Context changes and cancellation when returning to a dispatcher.

[13] [SupervisorJob API](#)

Direct-child independence and downward parent cancellation.

[29] [launch API](#) / [30] [Shared mutable state and concurrency](#).

[31] [Kotlin/JVM JUnit setup](#) / [32] [Mutex API](#).

Official reading / streams, Android, and tests

Primary references, linked for deeper study

[14] [StateFlow API](#)

Equality, conflation, initial state, and never-completing collection.

[15] [Android: StateFlow and SharedFlow](#)

Sharing policies and lifecycle-aware state collection.

[16] [Coroutine context and dispatchers](#)

Read context inheritance, dispatcher selection, and parent Jobs.

[17] [Composing suspending functions](#)

Compare sequential composition with structured concurrent async.

[18] [flow builder API](#)

Cold execution, cancellable emission, and context invariants.

[19] [flowOn API](#)

Upstream context changes and in-flight emissions on cancellation.

[20] [Flow catch API](#)

Upstream recovery and transparent downstream/cancellation behavior.

[21] [Android: coroutine best practices](#)

Main-safe suspend functions, dispatcher injection, scopes, and cancellation.

[22] [kotlinx-coroutines-test API](#)

runTest, TestScope, test schedulers, and replacing Main.

[23] [SharedFlow API](#)

Replay, buffering, and behavior with no subscribers.

[24] [runCatching API](#)

Observe the Throwable catch contract before applying it to suspend code.

[25] [Android: lifecycle-aware coroutines](#)

viewModelScope, lifecycleScope, and repeatOnLifecycle.

[26] [Android: testing Kotlin coroutines](#)

Practical tests with shared schedulers, dispatcher injection, and Main replacement.

[27] Operator APIs: [buffer](#), [conflate](#), [collectLatest](#), [combine](#), [zip](#), [retryWhen](#).

[28] [MutableStateFlow.update API](#): atomic updates and lambda reevaluation.

Documentation checked on 30 September 2026. API pages inspected show kotlinx.coroutines 1.11.0. AndroidX artifact versions should follow the target project's compatible version catalog. Code examples are teaching fixtures, not network-service implementations.